



## TEMA IV

### DESARROLLO DE APLICACIONES WEB EN BACKEND

*"Siempre parece imposible,  
hasta que se hace"*  
Nelson Mandela

#### 11. MICROSERVICIOS REST\_FRAMEWORK

Las arquitecturas software orientadas a microservicios se están convirtiendo en un standard para la construcción de aplicaciones modernas desplegadas en plataformas Cloud. Este estilo de arquitectura se está imponiendo en entornos donde las necesidades de escalabilidad son muy fuertes y el diseño en componentes independientes es una gran ventaja a la hora de abordar problemas complejos.

Esta arquitectura permite definir aplicaciones software mediante un conjunto de servicios desplegables de forma independiente, es decir, una aplicación es un conjunto de pequeñas aplicaciones poco acopladas.

La comunicación entre los servicios se ha de implementar a través de mecanismos ligeros, preferiblemente APIs basados en [HTTP](#), por ejemplo [RESTful/JSON](#).

Estos sistemas de comunicación, además, nos abren la puerta a que podamos desarrollar cada microservicio con el lenguaje y las herramientas que más nos convenga en cada caso. Por ejemplo, si en un microservicio concreto el rendimiento es un aspecto crítico, podemos escoger lenguajes de bajo nivel que nos permitan afinar más este aspecto como C, C++.

Sin embargo en otros casos nos interesará escoger lenguajes de alto o nivel o que implementen ciertas características particulares, como Python, que nos permitan avanzar más rápido o se ajusten mejor al servicio que deben implementar.

Por otro lado, en python, existen frameworks y bibliotecas muy asentados y orientados al desarrollo de APIs REST y sistemas de comunicación basados en HTTP, lo que es perfecto para la construcción de microservicios.

Algunos de los elementos importantes que tendremos que codificar son:

**Modelo:** modelo Película

**Vista:** funcionalidad que queremos ofrecer asociada en este microservicio. Hasta ahora hemos trabajado con vistas “basadas en funciones”, podemos trabajar también con vistas “basadas en clases” que se corresponden con los métodos de HTTP utilizados para CRUD:

- CreateAPIView → POST
- RetrieveAPIView y ListAPIView → GET
- UpdateAPIView → PUT + PATCH
- DestroyAPIView → DELETE

Las clases definen las variables `queryset` y `serializer_class`, para el acceso a la bbdd y el formato al serializar respectivamente

•**Router:** En este punto asociaremos la lógica definida en las vistas en rutas y métodos HTTP concretos. Generan automáticamente estructuras de URLs típicas

•**Serializador:** Es necesario definir el formato y estructura de los datos que recibe y devuelve nuestro API. Convierten objetos de Python a formatos de datos más simples como JSON y XML (serialización) y viceversa (deserialización). También validan los datos que recibe la aplicación, como los Forms en Django

Vamos a ver un ejemplo de un pequeño microservicio que exponga un API REST para manejar “películas” con Django y REST Framework.

Tenemos que instalar rest framework:

```
pip install djangorestframework
pip install markdown          # Markdown support for the browsable API.
pip install django-filter     # Filtering support
```